Singly linked list operations

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct LinkedList {
    struct Node* head;
    int size;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

struct LinkedList* initializeList() {
    struct LinkedList* list = (struct LinkedList*)malloc(sizeof(struct LinkedList));
    if (list == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
```

```
    list->head = NULL;
    list->size = 0;
    return list;
}

void insert(struct LinkedList* list, int data, int position) {
        if(position < 0 || position > list -> size){
                printf("Invalid position\n");
                        return;
        }
    struct Node* newNode = createNode(data);
        if(position == 0){
                newNode -> next = list -> head;
                list -> head = newNode;
        }else{
                struct Node* current = list -> head;
                for(int i = 0; i < position - 1; i++){
                        current = current -> next;
                }
                newNode -> next = current -> next;
                current -> next = newNode;
        }
        list -> size++;
        printf("Element %d inserted at position %d\n", data, position);
}
void deleteNode(struct LinkedList* list, int position) {
    if(position < 0 || position >= list -> size){
                printf("Invalid position\n");
                return;
        }
        struct Node* temp;
```

```
        if(position == 0){
                temp = list -> head;
                list -> head = temp -> next;
        }else{
                struct Node* current = list -> head;
                for(int i = 0; i < position - 1; i++){
                        current = current -> next;
                        current -> next = temp -> next;
                }
                temp = current -> next;
                current -> next = temp -> next;
        }
        free(temp);
        list -> size--;
        printf("Element at position %d deleted\n", position);
}
void reverse(struct LinkedList* list) {
   struct Node* prev = NULL;
        struct Node* current = list -> head;
        struct Node* next = NULL;
        while(current != NULL){
                next = current -> next;
                current -> next = prev;
                prev = current;
                current = next;
        }
        list -> head= prev;
}
void display(struct LinkedList* list) {
   struct Node* current = list->head;
   while (current != NULL) {
```

```
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

void displayReversed(struct LinkedList* list) {
    struct Node* current = list->head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

int main() {
    int num_elements;
    printf("Enter the number of elements in the linked list: ");
    scanf("%d", &num_elements);
    struct LinkedList* list = initializeList();
    int data;
    printf("Enter the elements:\n");
    for (int i = 0; i < num_elements; i++) {
        scanf("%d", &data);
        insert(list, data, i);
    }
    int choice, position;
    while (1) {
        printf("Linked List Operations:\n");
        printf("1. Insert\n");
        printf("2. Delete\n");
```

```
printf("3. Reverse\n");
printf("4. Display\n");
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
    case 1:
        printf("Enter data to insert: ");
        scanf("%d", &data);
        printf("Enter position to insert: ");
        scanf("%d", &position);
        insert(list, data, position);
        break;
    case 2:
        printf("Enter position to delete: ");
        scanf("%d", &position);
        deleteNode(list, position);
        break;
    case 3:
        reverse(list);
        printf("Reversed List: ");
        displayReversed(list);
        reverse(list);
        break;
    case 4:
        printf("Linked List: ");
        display(list);
        break;
    case 5:
        printf("Exiting program\n");
        exit(0);
```

```
        default:
            printf("Invalid choice\n");
    }
  }

  return 0;
}
```